

Experiment 3

Design of Lexical Analyzer to Recognize the Patterns of Strings using LEX

Aim

To design and implement a lexical analyzer using LEX (Flex) for recognizing keywords, identifiers, and integer constants.

Software Requirements

- Windows 10 / Windows 11
- Windows Terminal / Command Prompt
- WSL (Ubuntu)
- Flex
- GCC Compiler
- Visual Studio Code

Theory

A Lexical Analyzer is the first phase of a compiler. It reads the source program character by character and groups them into meaningful units called tokens.

Examples of tokens include:

- Keywords
- Identifiers
- Constants
- Operators
- Delimiters

LEX (Flex) accepts Regular Expressions as input and automatically generates a C program that performs lexical analysis.

LEX Workflow

Source Program



LEX Program (.l)



Flex



lex.yy.c



GCC Compiler



Executable Scanner

Explanation

The scanner recognizes:

Patter n	Token
int	Keyword

float	Keyword
char	Keyword
total	Identifier
numbe r123	Identifier
100	Integer Constant

Any character that does not match the above rules is reported as an Unknown Token.

Conclusion

In this experiment, students learned the basic structure of a LEX program and successfully generated a lexical analyzer using Flex. The experiment demonstrated how Regular Expressions can be used to recognize different token types, providing the foundation for the lexical analysis phase of a compiler.